Supplementary Material

Supplementary Figures

Supplementary Table

Supplementary Programs

Supplementary Figure 1

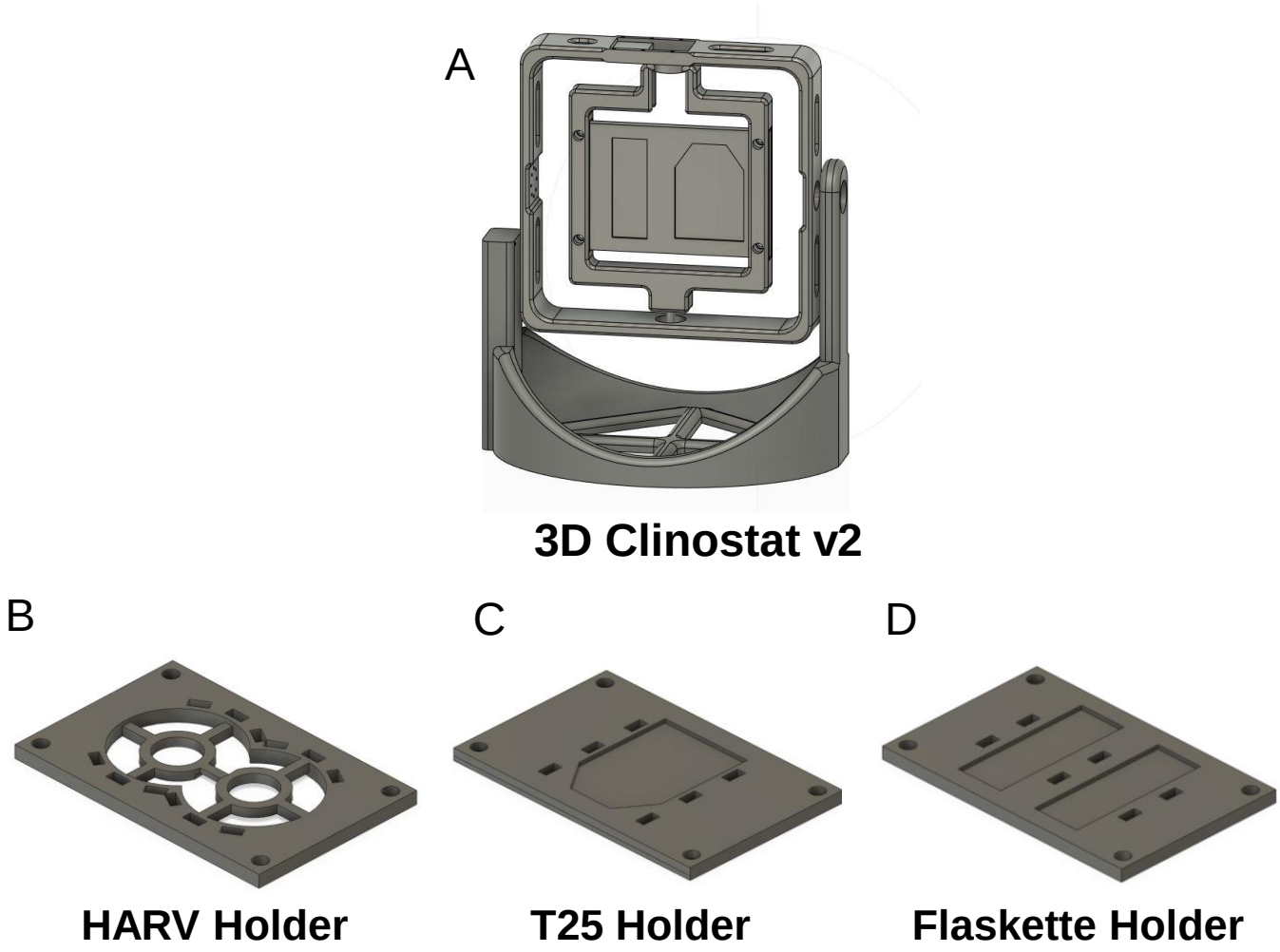

Figure 1 - CAD Model of 3D Clinostat v2 with sample holders
Fusion 360 was used to generate the designs for the 3D clinostat v2 (A), and the sample holders (B-D) that slide into the inner frame. The sample holder shown on the clinostat (A) can carry a cell culture flask with an area of 25 cm^2 (T25) and a flaskette on each side. It is necessary to balance the weight on each side of the holder. The HARV holder can carry two HARVs (B) and air flow to the membrane on the lower side of the HARV is not restricted due to the design of the holder. The T25 holder (C) can carry a T25 cell culture flask on each side of the holder, and the flaskette holder (D) can carry two flaskettes on each side. The samples are attached with zip ties using the holes in the holder.

Supplementary Figure 2

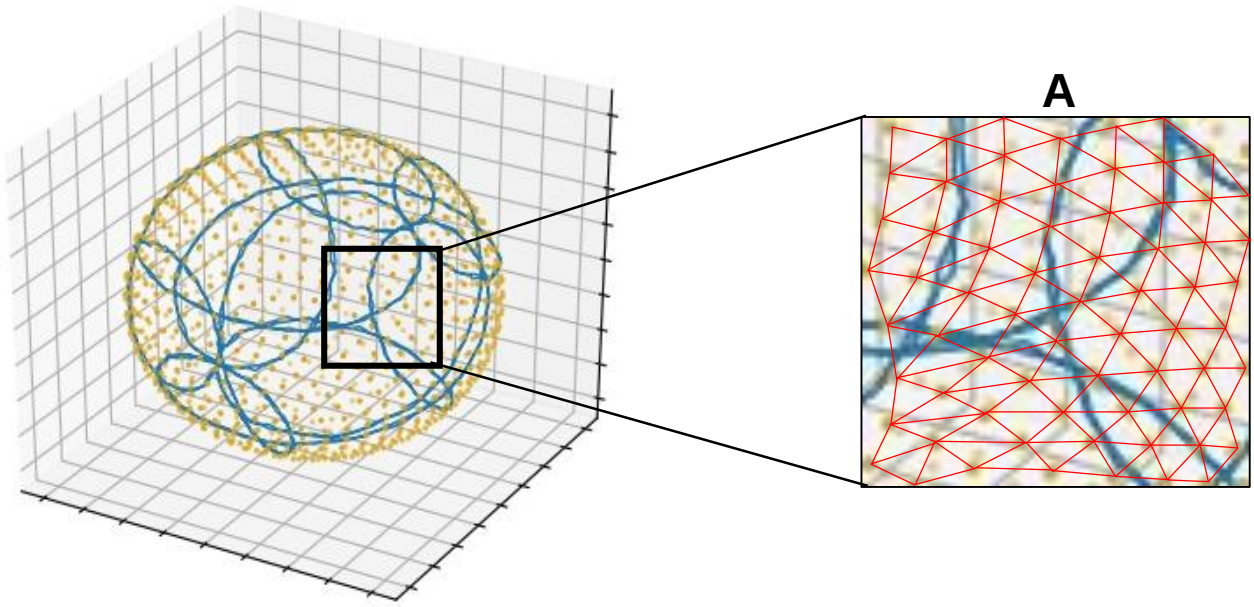

Figure 2 – Quantitating the Distribution of the Acceleration Vector
A Python program graphed the x, y, and z components of the acceleration vector and a program was developed to score how distributed the gravity path was around the unit sphere. The program evenly distributed 1000 points (yellow dots) around the unit sphere using a Fibonacci Lattice. For each acceleration vector position on the sphere (provided by the x, y and z), the program found the closest three points (yellow dots), effectively finding what triangular region the vector data point was located in (A). If the vector data point was the first to be located within that region, the distribution score was increased by one, otherwise, the score remained the same.

Supplementary Figure 3

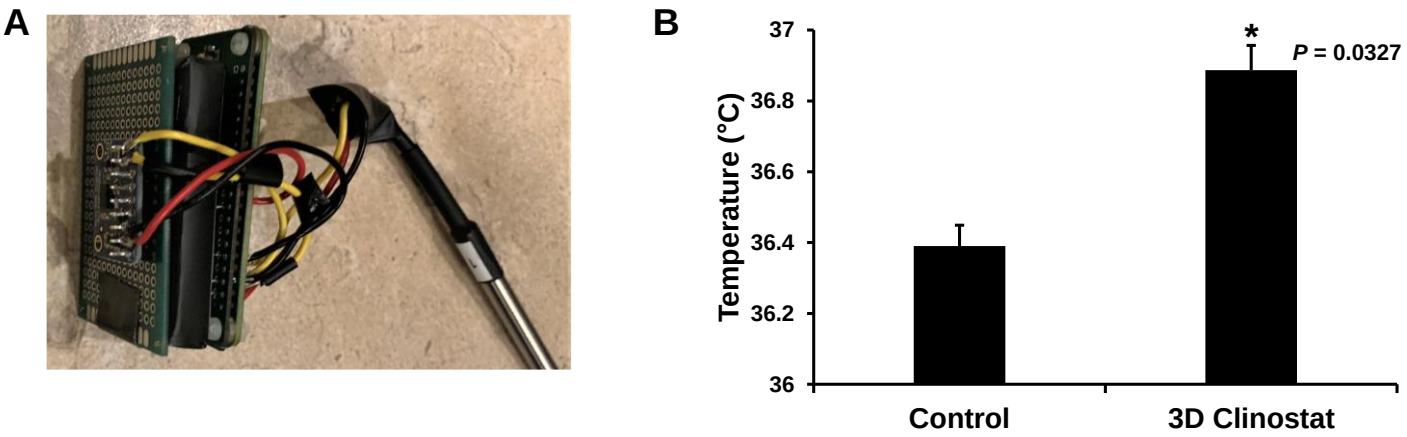

Figure 3 - Incubator Temperature Change Due to Clinostat Operation
The 3D clinostat v1 was placed in a 37°C cell culture incubator. The temperature probe attached to a Raspberry Pi zero and a Pi Sugar battery (A) was placed in the incubator, and the incubator allowed to reach operating temperature. The temperature was monitored for 48 hours with either the 3D clinostat inactive (Control, B) or with the 3D clinostat operating with inner and outer rings at 0.625 rpm and 1.25 rpm, respectively. The average temperature was calculated from the data collected. The experiment was repeated 3 times, and the average temperature and the standard deviation calculated (B).

Supplementary Figure 4

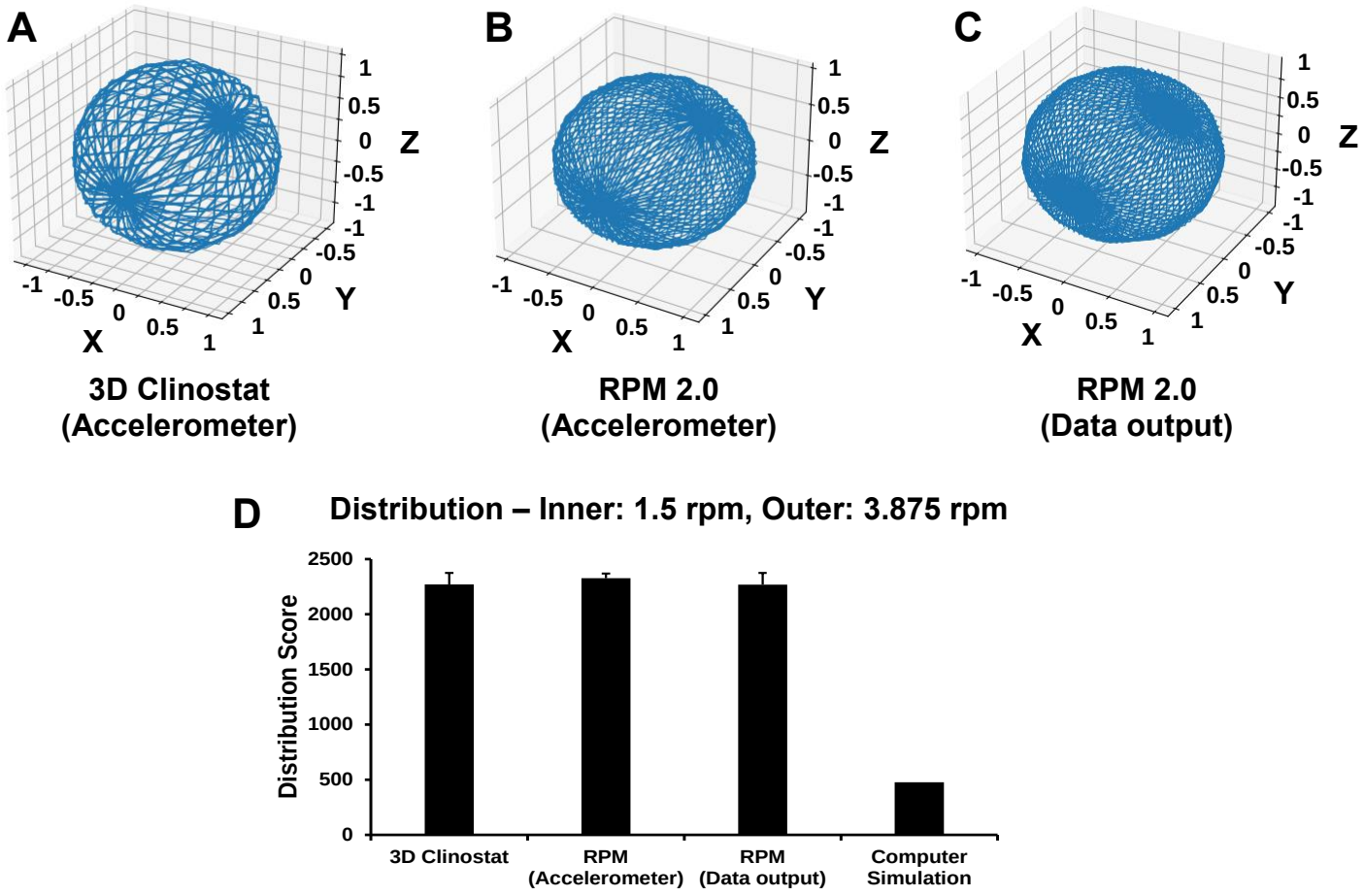

Figure 4 – Path and Distribution of Acceleration Vector
An Arduino UNO WIFI Rev 2, which has an onboard accelerometer, was attached to the 3D clinostat and the RPM 2.0. The acceleration vector path was plotted using data obtained from the Arduino accelerometer with the machines operating at an inner frame and outer frame velocity combination of 1.5 rpm and 3.875 rpm, respectively (A and B). The data output by the algorithm from the RPM 2.0 was also plotted (C). The distribution score was calculated for these frame velocities and is shown in D. Data collected from 0-3000 seconds was used to calculate the distribution score. Path images (A-C) were from data collected between 2000-3000 seconds so the pattern could be seen. The conditions were performed three times and the average and standard deviation shown.

Supplementary Figure 5

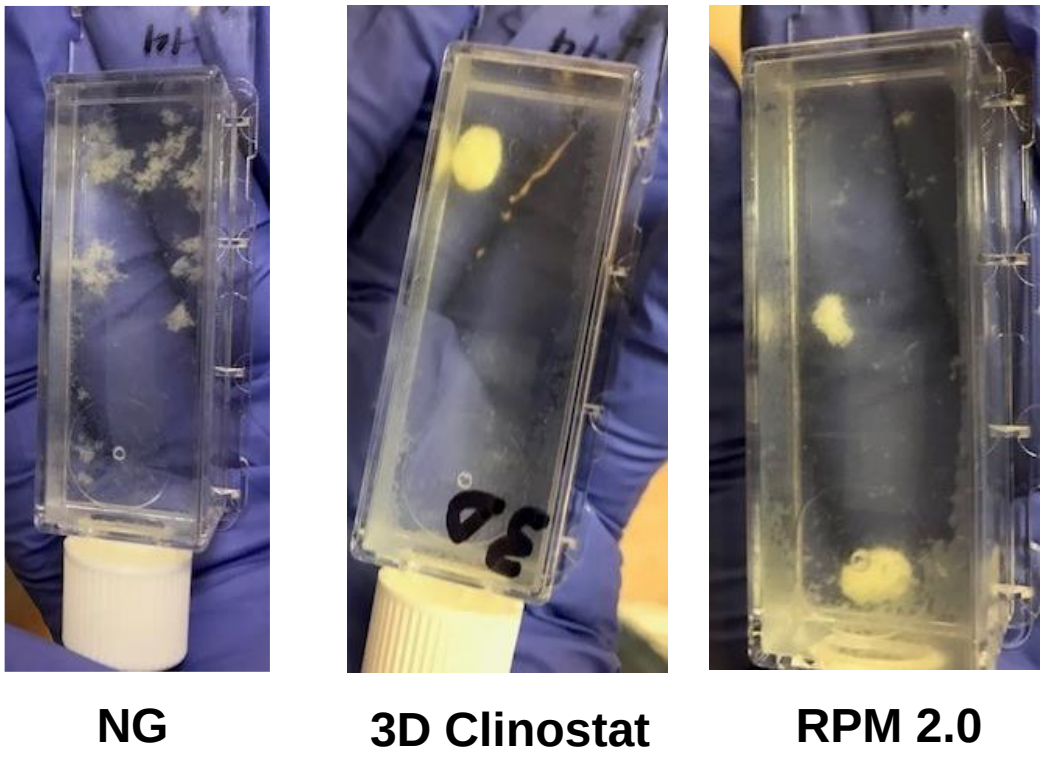

Figure 5 – *M. marinum* form a mass of cells when exposed to simulated microgravity using the 3D clinostat v2 or RPM 2.0
Flaskettes were filled with bacteria in biofilm-promoting medium and attached to the RPM 2.0 or 3D clinostat. The RPM 2.0 was operated using the partial gravity mode and 0g file, and the 3D clinostat was operated with an inner frame velocity of 1.5 rpm and outer frame velocity of 3.875 rpm. Control flasks for normal gravity (NG) were placed on a shelf in the same incubator that housed the 3D clinostat and RPM 2.0. After four days at 31ºC, the flaskettes were removed and the images captured. The bacteria were in a mass when removed from the 3D clinostat and the RPM 2.0, but were dispersed aggregates in the NG flask.

Supplementary Figure 6

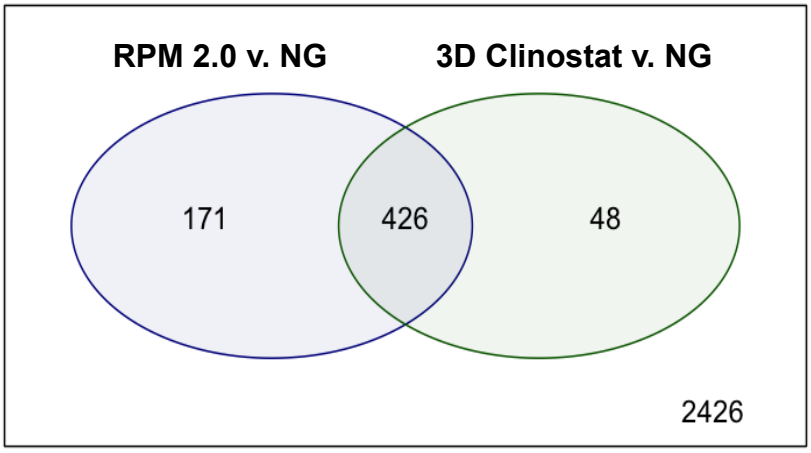

Figure 6 – Venn diagram showing the overlap of gene expression changes induced by growth of *M. marinum* when exposed to simulated microgravity using the RPM 2.0 and 3D clinostat
Differential expression analysis for 3071 genes was performed on RNA-Seq data from RNA isolated from *M. marinum* grown in biofilm-promoting medium for 4 days under normal gravity (NG) or simulated microgravity on the RPM 2.0 or the 3D clinostat (I: 1.5 rpm O: 3.875 rpm). Comparisons of the differential analyses for RPM 2.0 and NG, and the 3D Clinostat and NG demonstrated an overlap of 426 genes.

Supplementary Figure 7

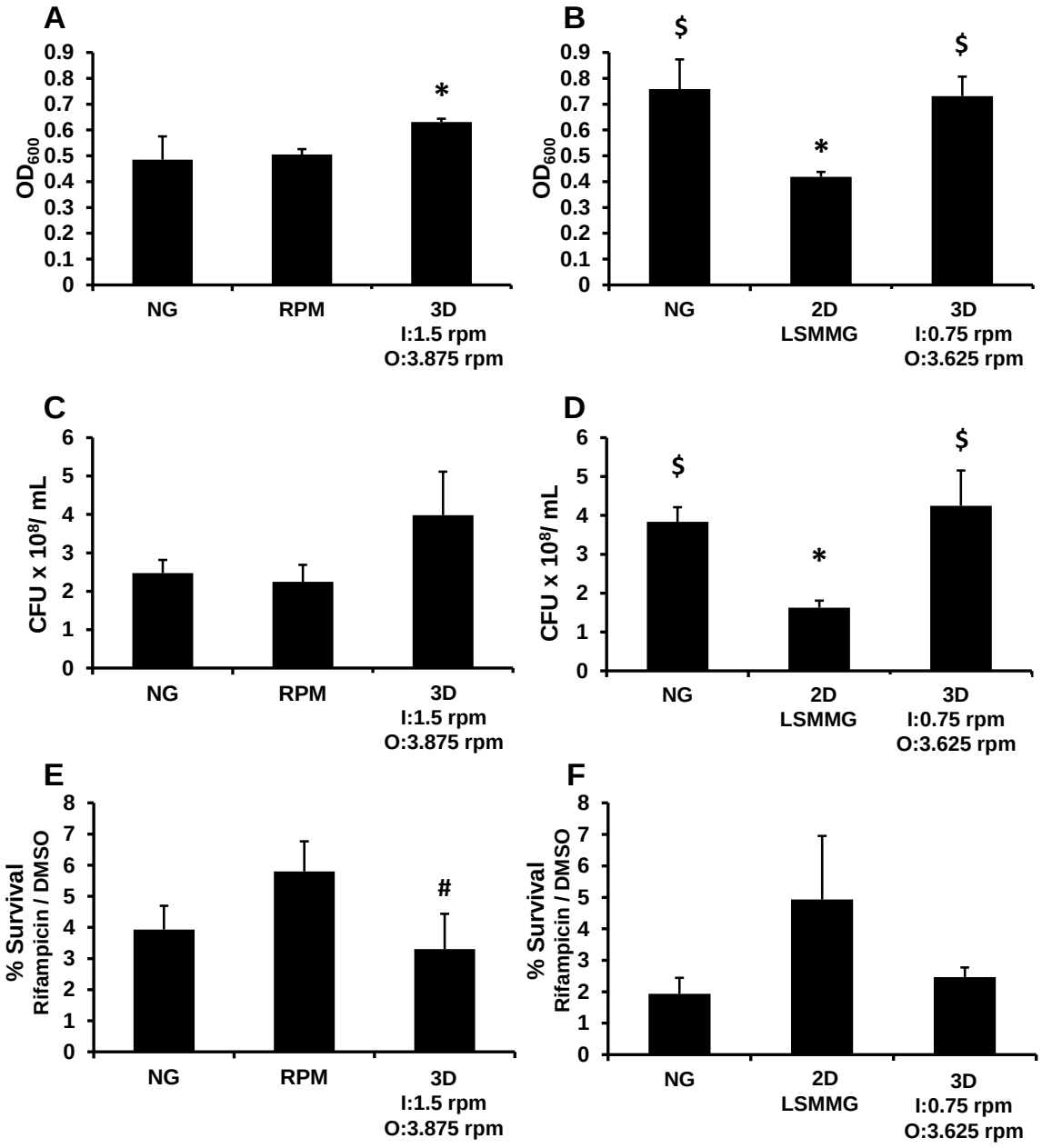

Figure 7 –Twenty-Four-Hour *M. marinum* Growth and Rifampicin Survival
M. marinum were grown in 10 mL HARVs in medium to promote planktonic growth with or without 0.8 µg/ mL rifampicin for 24 hrs at 31ºC. The RPM 2.0 (RPM) was operated in partial gravity mode using the 0 g file, and the 2D clinostat from Synthecon was used for the normal gravity (NG) and LSMMG conditions at 25 rpm. For the 3D clinostat, I = inner frame velocity in rpm and O = outer frame velocity in rpm. Work was performed at Kennedy Space Center in A, C and E and at LSUHS for B, D and F. The OD_{600} (A and B) and the CFU/ mL (C and D) were used to compare growth without rifampicin. The percentage of *M. marinum* surviving the 24-hr rifampicin treatment is shown in D and E. Experiments were performed three times and the average and standard deviation is shown. * $p < 0.05$ compared to NG, # $p < 0.05$ compared to RPM, $ $p < 0.05$ compared to LSMMG

Supplementary Figure 8

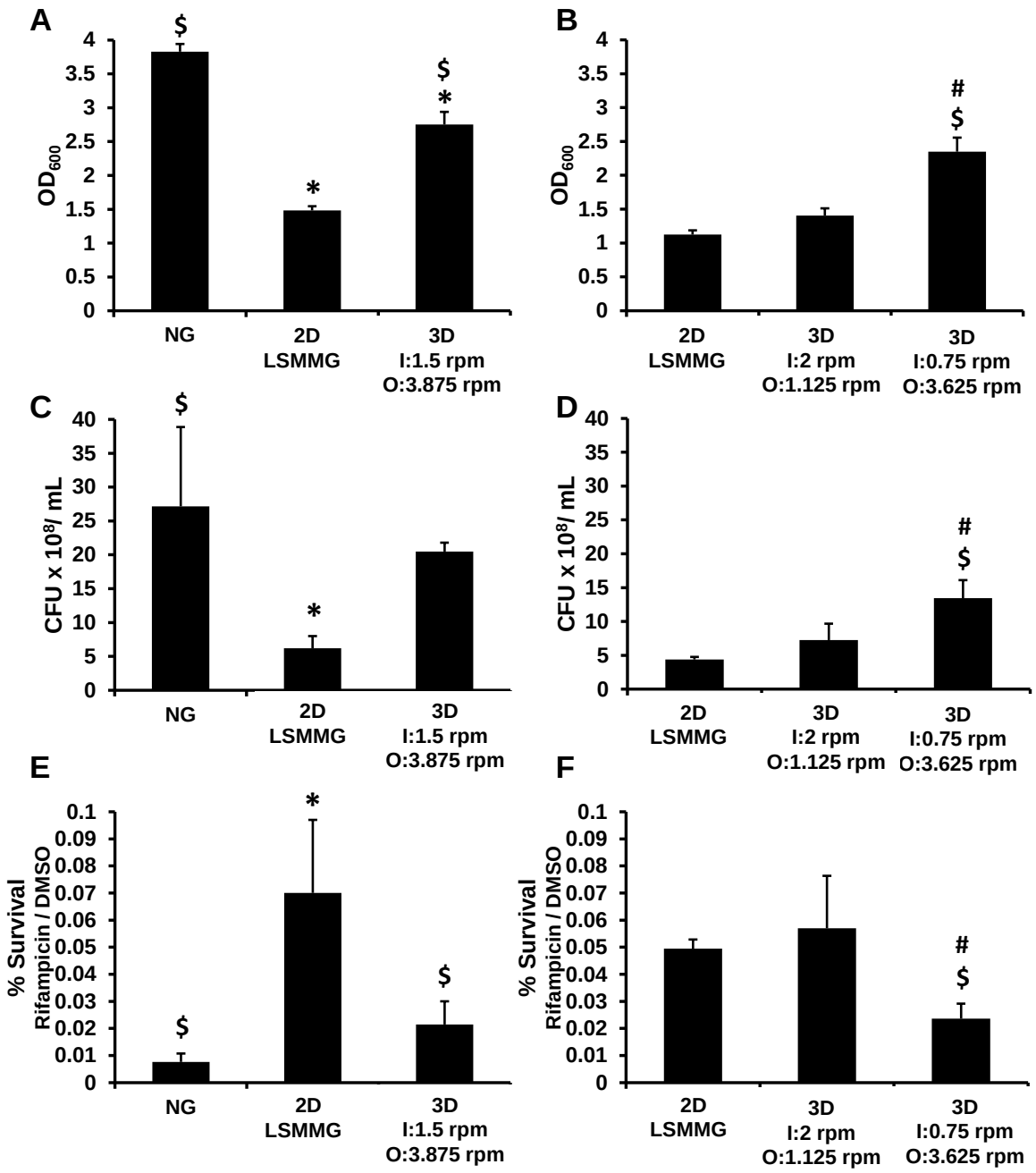

Figure 8 –Four-Day *M. marinum* Growth and Rifampicin Survival
M. marinum were grown in 10 mL HARVs in medium to promote planktonic growth with or without 0.05 µg/ mL rifampicin for 4 days at 31ºC. The 2D clinostat from Synthecon operated at 25 rpm for the NG and LSMMG conditions. For the 3D clinostat, I = inner frame velocity in rpm and O = outer frame velocity in rpm. All work was performed at LSUHS. Experiments in A, C and E used the 4 unit Synthecon 2D clinostat for the NG and LSMMG cultures. Due to the size of the units, the NG and LSMMG cultures were grown in a different incubator to the 3D clinostat culture. Experiments in B, D and F used two single unit Synthecon 2D clinostats for LSMMG and the cultures were grown in the same incubator as the 3D clinostat culture. The OD_{600} (A and B) and CFU/ mL (C and D) are shown for cultures without rifampicin. The percentage of *M. marinum* surviving the 4-day rifampicin treatment is shown in D and E. Experiments were performed three times and the average and standard deviation is shown. * $p < 0.05$ compared to NG, $ $p < 0.05$ compared to LSMMG, # $p < 0.05$ compared to I:2 rpm O: 1.125 rpm

Supplementary Figure 9

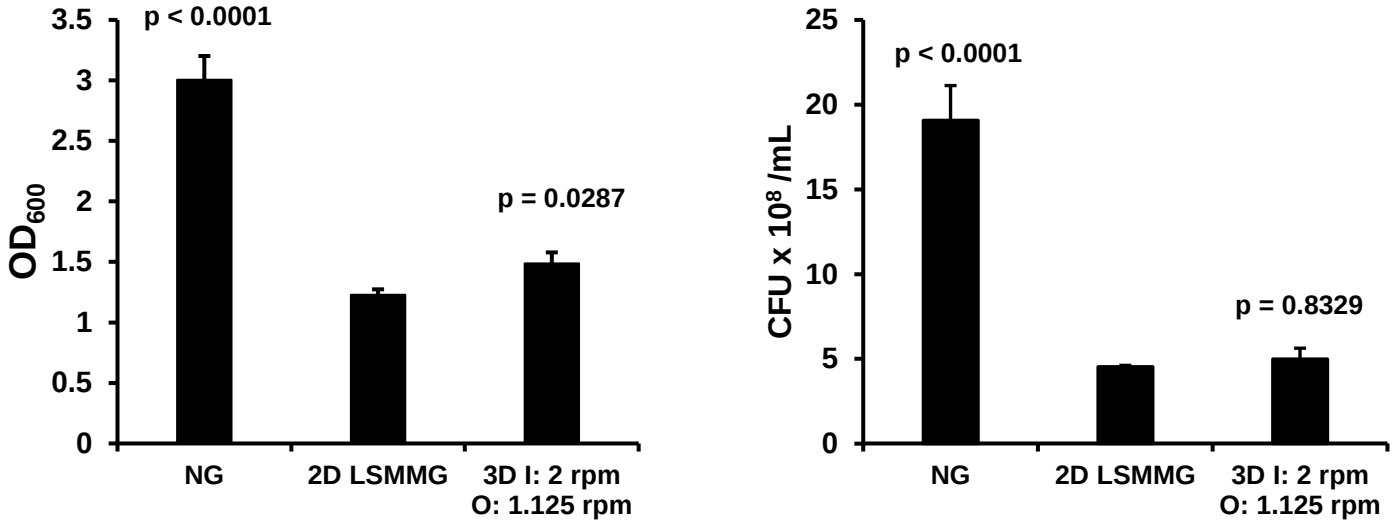

Figure 9 –Four day *M. marinum* growth
M. marinum were grown in 10 mL HARVs in medium to promote planktonic growth for 4 days at 31ºC. The 2D clinostat from Synthecon was operated at 25 rpm for the normal gravity (NG) and LSMMG conditions. For the 3D clinostat, I = inner frame velocity in rpm and O = outer frame velocity in rpm. Experiments were performed three times and the average and standard deviation is shown for the OD_{600} (A) and the CFU/ mL (B). The p values are in comparison to the 2D LSMMG culture.

Table 1 - Number of genes with altered transcript levels due to simulated microgravity
The rRNA genes were removed from the RNA-seq data before performing the analyses below. Genes with low expression were removed if the minimum count was less than 10 (**A** and **B**) or 5 (**C** and **D**) in approximately 70% of the samples, and the minimum total count of all samples had to equal at least 15. In **E**, low expression genes were not filtered from the analysis.

A: The outlier 3D clinostat sample was included in the analysis. Minimum count less than 10.

Type of Gene expression change	Type of Comparison		
	3D clinostat v. RPM 2.0	**RPM 2.0 v. NG**	**3D clinostat v. NG**
Decreased expression	0	350	292
No change	3071	2474	2597
Increased expression	0	247	182

B: The outlier 3D clinostat sample was excluded from the analysis. Minimum count less than 10.

Type of Gene expression change	Type of Comparison		
	3D clinostat v. RPM 2.0	**RPM 2.0 v. NG**	**3D clinostat v. NG**
Decreased expression	0	393	319
No change	3071	2405	2530
Increased expression	0	273	222

C: The outlier 3D clinostat sample was included in the analysis. Minimum count less than 5.

Type of Gene expression change	Type of Comparison		
	3D clinostat v. RPM 2.0	**RPM 2.0 v. NG**	**3D clinostat v. NG**
Decreased expression	0	383	320
No change	3925	3281	3414
Increased expression	0	261	191

D: The outlier 3D clinostat sample was excluded from the analysis. Minimum count less than 5.

Type of Gene expression change	Type of Comparison		
	3D clinostat v. RPM 2.0	**RPM 2.0 v. NG**	**3D clinostat v. NG**
Decreased expression	0	428	332
No change	3925	3216	3385
Increased expression	0	281	208

E: The outlier 3D clinostat sample was excluded from the analysis. No filtering for low gene expression.

Type of Gene expression change	Type of Comparison		
	3D clinostat v. RPM 2.0	**RPM 2.0 v. NG**	**3D clinostat v. NG**
Decreased expression	0	480	374
No change	5557	4839	5016
Increased expression	0	238	167

Supplementary Program 1 – Clinostat Control (Arduino C)

```
#include <ax12.h>
#include <BioloidController.h>

void setup()
{
  //Set motor to wheel mode
  ax12SetRegister2(3, AX_CCW_ANGLE_LIMIT_L, 0); //inner
  ax12SetRegister2(2, AX_CCW_ANGLE_LIMIT_L, 0); //outer
  delay(1000);
}

void loop()
{
  ax12SetRegister2(2, AX_GOAL_SPEED_L, 50); //inner
  ax12SetRegister2(3, AX_GOAL_SPEED_L, 4); //outer
  Serial.println(inner);
  delay(250);
}

```

Supplementary Program 2 – Arduino UNO Accelerometer (Arduino C)

```
#include <ArduinoHttpClient.h>
#include <WiFiNINA.h>
#include <Arduino_LSM6DS3.h>
#include "arduino_secrets.h"

//Set Wifi credentials in arduino_secrets.h
//Wifi Settings
char ssid[] = SECRET_SSID;
char pass[] = SECRET_PASS;

char serverAddress[] = "192.168.1.1"; //SET RECEIEVING COMPUTER IP ADDRESS
int port = 8080;

WiFiClient wifi;
WebSocketClient client = WebSocketClient(wifi, serverAddress, port);
int status = WL_IDLE_STATUS;

int failCount = 0;

unsigned long t;


void setup() {
  Serial.begin(9600);
  wifi_setup();

  //Setup for Accelerometer
  if (!IMU.begin()){
    Serial.println("Failed to initialize IMU!");
    while (1);
  }

  Serial.print("Accelerometer sample rate = ");
  Serial.print(IMU.accelerationSampleRate());
  Serial.println(" Hz");
  Serial.println();
  Serial.println("Acceleration in G's");
  Serial.println("X\tY\tZ");
}

void wifi_setup() {
  //Setup for WIFI
  while (status != WL_CONNECTED) {
    Serial.println(status);
    Serial.print("Attempting to connect to Network named: ");
    Serial.println(ssid);                   // print the network name (SSID);

    // Connect to WPA/WPA2 network:
    status = WiFi.begin(ssid, pass);
  }

  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your WiFi shield's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);
```

```
}

void loop() {
  float x,y,z;
  int messageSize;

  Serial.println("Starting WebSocket client");
  client.begin();
  Serial.println("Websocket Connected");

  while (client.connected()) {
    t = millis();
    if (IMU.accelerationAvailable()) {
      IMU.readAcceleration(x,y,z);

      client.beginMessage(TYPE_TEXT);
      client.print(x);
      client.print('\t');
      client.print(y);
      client.print('\t');
      client.print(z);
      client.print('\t');
      client.println(t);
      client.endMessage();

      Serial.print(x);
      Serial.print('\t');
      Serial.print(y);
      Serial.print('\t');
      Serial.println(z);
      messageSize = client.parseMessage();

      if (messageSize == 0) {
        failCount++;
        Serial.print("Fail: ");
        Serial.println(failCount);
      }
      else {
        failCount = 0;
      }

      if (failCount > 10) {
        break;
      }

    }
    delay(1000);
  }
  client.stop();
  wifi_setup();
  client.println("disconnected\n");
  Serial.println("disconnected\n");
}

```

Supplementary Program 3 - Accelerometer Web Socket (Java Processing)

```
import websockets.*;

WebsocketServer ws;
int now;
int count = 0;
int old_count = 0;
int failCount = 0;
float x, y;
String old_msg;
PrintWriter output;

String path = "C:/OUTPATH/"; //SET OUTPUT PATH HERE
String name = path + "FILENAME.txt"; //SET OUTPUT FILE NAME HERE

void setup() {
  size(200, 200);
  ws = new WebsocketServer(this, 8080, "");

  output = createWriter(name);
  now=millis();
  x=0;
  y=0;
}

void draw() {
  //Debug display
  ellipse(x, y, 10, 10);

  if (millis()>now+1000) {
    if(old_count == count) {
      print("Fail ");
      print(failCount);
      print("/");
      println(count);
      output.print("fail: ");
      output.println(millis());
      output.flush();
      background(random(255), random(255), random(255));
      failCount++;
    }
    else {
      ws.sendMessage("Success");
      old_count = count;
    }

    now = millis();
  }
}

void webSocketServerEvent(String msg) {
  //Triggered when data received
  //Outputs data to file that can be parsed by python program
  count++;
  println(msg);
  output.println(msg);
  output.flush();
  x=random(width);
  y=random(height);
}
```

Supplementary Program 4 – Computer Model (Python)

```python
import math

class Sim:
    #Returns x-component of the G vector at a specific time for a specific inner and outer frame angular velocities
    def gVectorX(self, timeInSeconds, localInnerInRadSec, localOuterInRadSec):
        ret = math.sin(localOuterInRadSec*timeInSeconds)*math.cos(localInnerInRadSec*timeInSeconds)
        return ret
    #Returns z-component of the G vector at a specific time for a specific outer frame angular velocity
    def gVectorY(self, timeInSeconds, localOuterInRadSec):
        ret = math.cos(localOuterInRadSec*timeInSeconds)
        return ret
    #Returns y-component of the G vector at a specific time for a specific inner and outer frame angular velocities
    def gVectorZ(self, timeInSeconds, localInnerInRadSec, localOuterInRadSec):
        ret = math.sin(localOuterInRadSec*timeInSeconds)*math.sin(localInnerInRadSec*timeInSeconds)
        return ret

    #Converts and returns frame velocity from RPM to radians per second
    def RPMtoRadSec(self, RPM):
        ret = RPM * (math.pi / 30)
        return ret

    #Returns a Python dictionary containing value of g vector of each axis at specific times
    def gVectorData(self, startTimeInSeconds, endTimeInSeconds, innerRPM, outerRPM):
        innerInRadSec = self.RPMtoRadSec(innerRPM)
        outerInRadSec = self.RPMtoRadSec(outerRPM)
        timeArray, xArray, yArray, zArray = [], [], [], []
        for t in range(startTimeInSeconds, endTimeInSeconds + 1):
            timeArray.append(t)
            xArray.append(self.gVectorX(t, innerInRadSec, outerInRadSec))
            yArray.append(self.gVectorY(t, outerInRadSec))
            zArray.append(self.gVectorZ(t, innerInRadSec, outerInRadSec))
        data = timeArray, xArray, yArray, zArray
        return data

```

Supplementary Program 5 - Data Processing and Visualization (Python)

```python
import os
import sys
import numpy as np
from pathVisualization import PathVisualization
from simulation import Sim

class DataProcessor:
    def __init__(self):
        #Uncomment machine type based on data source
        self.machineType = "sim"
        #self.machineType = "clinostat"
        #self.machineType = "rpm-integ"
        #self.machineType = "rpm-ard"

        #Set range for average magnitude (seconds)
        self.minSeg = 2000
        self.maxSeg = 3000

        #Set data output path
        outParent = "C:\\OUTPATH\\" #SET OUTPUT FILE PATH

        if self.machineType != "sim":
            self.inPath =  "C:\\INPATH\\" #SET INPUT FILE PATH
            self.inName = "exampleFile.txt" #SET FILE NAME

            try:
                testOpen = open(self.inPath + self.inName, "r")
                testOpen.close()
            except FileNotFoundError:
                print("\nERROR: Input file not found: " + self.inPath + self.inName + "\n")
                sys.exit()
            if self.machineType != "sim":
                self.outPath = outParent + self.inName[:len(self.inName)-4] + "\\"
            else:
                self.outPath = outParent + self.inName + "\\"
            print(self.outPath)
        else: #Applies only for Computer Model
            self.innerV = 3.875 #SET INNER VELOCITY
            self.outerV = 1.5 #SET OUTER VELOCITY
            self.inName = str(self.innerV) + "-" + str(self.outerV)
            self.outPath = outParent + self.inName + "\\"

        if os.path.isdir(self.outPath) == False:
            os.makedirs(self.outPath)

    def outputDataFile(self):
        """Outputs processed data file"""
        self.__getVectors()
        xTimeAvg, yTimeAvg, zTimeAvg = self.__getTimeAvg()
        magList = self.__getMagnitude(xTimeAvg, yTimeAvg, zTimeAvg)
        avgMagSeg = self.__getMagSeg(magList)
        disScore = self.__getDisScore()

        fileOut = open(self.outPath + 'processedAccelData.txt', 'w+')

        if self.machineType != 'sim':
            fileOut.write(self.inName)
        else:
```

```
59              fileOut.write('Inner: ' + str(self.innerV) + '\t' + 'Outer: ' +
    str(self.outerV) + '\n')
60
61          fileOut.write('Segment: ' + str(self.minSeg) + ":" + str(self.maxSeg) + '\t'
    + str(avgMagSeg) + '\t' + '\t\t\t' + 'Dispersion: ' + '\t' + str(disScore) +'\n')
62          fileOut.write('Time' + '\t' + 'X Vector' + '\t' + 'Y Vector' + '\t' + 'Z
    Vector' + '\t' + 'X Time Avg' + '\t' + 'Y Time Avg' + '\t' + 'Z Time Avg' + '\t' +
    'Magnitude' + '\n')
63
64          for i in range(len(self.x)):
65              fileOut.write(str(self.time[i]) + '\t' + str(self.x[i]) + '\t' +
    str(self.y[i]) + '\t' + str(self.z[i]) + '\t' + str(xTimeAvg[i]) + '\t' +
    str(yTimeAvg[i]) + '\t' + str(zTimeAvg[i]) + '\t' + str(magList[i]) + '\n')
66
67          fileOut.close()
68
69          print("Average Magnitude: " + str(avgMagSeg))
70          print("Distribution: " + str(disScore))
71
72      def __getVectors(self):
73          """Directs where to get data from"""
74          if self.machineType == 'sim':
75              self.time, self.x, self.y, self.z = self.__getSimAccelData()
76          elif self.machineType == 'clinostat' or self.machineType == 'rpm-ard':
77              self.time, self.x, self.y, self.z = self.__getArdAccelData()
78          else:
79              self.time, self.x, self.y, self.z = self.__getRPMAccelData()
80
81      def __getArdAccelData(self):
82          """Reads data from file formatted by the Arduino UNO Accelerometer"""
83          dataIn = open(self.inPath + self.inName, 'r')
84
85          x, y, z, time = [], [], [], []
86          for line in dataIn.readlines():
87              lineSplit = line.split('\t')
88              if len(lineSplit) == 4:
89                  x.append(float(lineSplit[0]))
90                  y.append(float(lineSplit[1]))
91                  z.append(float(lineSplit[2]))
92                  unTime = float(lineSplit[3]) / 1000
93                  time.append(unTime)
94
95          dataIn.close()
96          return time, x, y, z
97
98      def __getRPMAccelData(self):
99          """Reads data from file formatted by the RPM 2.0"""
100         dataIn = open(self.inPath + self.inName, 'r')
101
102         x, y, z, time = [], [], [] ,[]
103         for line in dataIn.readlines():
104             lineFirstSplit = line.split('\t')
105             splitVectors = lineFirstSplit[1]
106             vectorList = splitVectors.split('   ')
107
108             x.append(float(vectorList[0]))
109             z.append(float(vectorList[1]))
110             y.append(float(vectorList[2]))
111
112         for i in range(len(x)):
```

```
            time.append(i)

        dataIn.close()
        return time, x, y, z

    def __getSimAccelData(self):
        """Gets data from computer model"""
        simInnerV = float(self.innerV)
        simOuterV = float(self.outerV)
        vectorSim = Sim()
        time, x, y, z = vectorSim.gVectorData(0, self.maxSeg, simInnerV, simOuterV)
        return time, x, y, z

    def __getTimeAvg(self):
        """Calculates Time Average"""
        xTimeAvg = []
        xTempList = []
        for xIter in self.x:
            xTempList.append(xIter)
            xTimeAvg.append(np.mean(xTempList))

        yTimeAvg = []
        yTempList = []
        for yIter in self.y:
            yTempList.append(yIter)
            yTimeAvg.append(np.mean(yTempList))

        zTimeAvg = []
        zTempList = []
        for zIter in self.z:
            zTempList.append(zIter)
            zTimeAvg.append(np.mean(zTempList))

        return xTimeAvg, yTimeAvg, zTimeAvg

    def __getMagnitude(self, xTimeAvg, yTimeAvg, zTimeAvg):
        """Calculates magnitude of time-averaged acceleration vector"""
        magList = []

        for i in range(len(self.x)):
            xIter = xTimeAvg[i]
            yIter = yTimeAvg[i]
            zIter = zTimeAvg[i]

            mag = (xIter ** 2 + yIter ** 2 + zIter ** 2) ** 0.5
            magList.append(mag)

        return magList

    def __getMagSeg(self, magList):
        """Calculates avergae magnitude between segmentd"""
        magSegList = magList[self.minSeg:self.maxSeg]
        if len(magList) < self.minSeg:
            print("\nERROR: Segment begins after data ends - " + str(len(magList)) +
" sec\n")
            sys.exit()
        elif len(magSegList) < (self.maxSeg - self.minSeg):
            print("\nWARNING: Not enough data for segment - " + str(len(magList)) + "
sec\n")
        return np.mean(magList[self.minSeg:self.maxSeg])
```

```
171
172     def __getDisScore(self):
173         xSeg = self.x[:self.maxSeg]
174         ySeg = self.y[:self.maxSeg]
175         zSeg = self.z[:self.maxSeg]
176         path = PathVisualization(self.inName, xSeg, ySeg, zSeg)
177         disScore = path.getDistribution()
178 
179         return disScore
180 
181     def copyRawData(self):
182         if self.machineType == "sim":
183             print("\nWARNING: Machine Type = Sim - No files were copied\n")
184             return
185         dataIn = open(self.inPath + self.inName, 'r')
186         copyData = open(self.outPath + self.inName[:len(self.inName)-4] + '-Copy' +
self.inName[len(self.inName)-4:], 'w+')
187 
188         copyData.write(self.inPath + self.inName + '\n\n')
189 
190         for line in dataIn.readlines():
191             copyData.write(line)
192 
193         copyData.close()
194         dataIn.close()
195 
196     def createGraphs(self):
197         self.__getVectors()
198         xTimeAvg, yTimeAvg, zTimeAvg = self.__getTimeAvg()
199         magnitude = self.__getMagnitude(xTimeAvg, yTimeAvg, zTimeAvg)
200 
201         if self.machineType != 'sim':
202             graphPath = PathVisualization(self.inName, self.x, self.y, self.z,
saveFile=self.outPath + self.inName[:len(self.inName)-4])
203         else:
204             graphPath = PathVisualization(self.inName, self.x, self.y, self.z,
saveFile=self.outPath + self.inName)
205 
206         graphPath.createPathShadowFig(mode='save', legend=False)
207 
208         graphPath.createVectorFig(self.time, mode='save')
209         graphPath.createTimeAvgFig(xTimeAvg, yTimeAvg, zTimeAvg, self.time,
mode='save')
210         graphPath.createMagFig(magnitude, self.time, mode='save')
211 
212     def createSegGraphs(self):
213         self.__getVectors()
214         xSeg = self.x[:self.maxSeg]
215         ySeg = self.y[:self.maxSeg]
216         zSeg = self.z[:self.maxSeg]
217 
218         if self.machineType != 'sim':
219             comparePaths = PathVisualization(self.inName, xSeg, ySeg, zSeg,
saveFile=self.outPath + self.inName[:len(self.inName)-4] + '-Segment')
220         else:
221             comparePaths = PathVisualization(self.inName, xSeg, ySeg, zSeg,
saveFile=self.outPath + self.inName + '-Segment')
222         comparePaths.createPathFig(mode='save')
223 
224 if __name__ == "__main__":
```

```
225     process = DataProcessor()
226     process.outputDataFile()
227     process.createGraphs()
228     process.createSegGraphs()
229     process.copyRawData()
230
231
```

Supplementary Program 6 - Accelerometer Vector Path Visualization (Python)

```python
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from mpl_toolkits.mplot3d import Axes3D

class PathVisualization:
    def __init__(self, ID, x, y, z, saveFile=''):
        self.ID = ID

        self.x = x
        self.y = y
        self.z = z

        self.pathCoords = list(zip(self.x, self.y, self.z))
        self.num_points = 1000

        self.saveFile = saveFile

    def __createSphere(self):
        """creates guide sphere with fibonnaci sequence on same plot as path"""
        golden_r = (np.sqrt(5.0) + 1.0) / 2.0            #golden ratio = 1.61803...
        golden_a = (2.0 - golden_r) * (2.0 * np.pi)      #golden angle + 2.39996...

        Xs, Ys, Zs = [], [], []

        for i in range(self.num_points):
            ys = 1 - (i / float(self.num_points - 1)) * 2
            radius = np.sqrt(1 - ys * ys)

            theta = golden_a * i

            xs = np.cos(theta) * radius
            zs = np.sin(theta) * radius

            Xs.append(xs)
            Ys.append(ys)
            Zs.append(zs)

        return(Xs, Ys, Zs)

    def __splitSphere(self, sphereCoords):
        octants = {'posI':[], 'posII':[], 'posIII':[], 'posIV':[], 'negI':[],
'negII':[], 'negIII':[], 'negIV':[]}

        for row in sphereCoords:
            #Coordinates: x = row[0], y = row[1], z = row[2]
            if (row[2] > 0):
                if (row[1] > 0):
                    if (row[0] > 0):
                        octants['posI'].append(row)
                    else:
                        octants['posII'].append(row)
                elif (row[0] > 0):
                    octants['posIV'].append(row)
                else:
                    octants['posIII'].append(row)
            else:
                if (row[1] > 0):
                    if (row[0] > 0):
```

```
                        octants['negI'].append(row)
                    else:
                        octants['negII'].append(row)
                elif (row[0] > 0):
                    octants['negIV'].append(row)
                else:
                    octants['negIII'].append(row)

        return(octants)

    def __getPathOctant(self, pathRow):
        """Checks what quadrant the tuple of coordinates is in"""
        if (pathRow[2] > 0):
            if (pathRow[1] > 0):
                if (pathRow[0] > 0):
                    return 'posI'
                else:
                    return 'posII'
            elif (pathRow[0] > 0):
                return 'posIV'
            else:
                return 'posIII'
        else:
            if (pathRow[1] > 0):
                if (pathRow[0] > 0):
                    return 'negI'
                else:
                    return 'negII'
            elif (pathRow[0] > 0):
                return 'negIV'
            else:
                return 'negIII'

    def __getDistanceBetween(self, pathTupleCoords, sphereTupleCoords):
        """Calculates distance between two points (path point and sphere point"""
        pathX, pathY, pathZ = pathTupleCoords
        sphereX, sphereY, sphereZ = sphereTupleCoords

        diffX = pathX - sphereX
        diffY = pathY - sphereY
        diffZ = pathZ - sphereZ

        sumSquares = diffX ** 2 + diffY ** 2 + diffZ ** 2
        dist = np.sqrt(sumSquares)

        return dist

    def __getDistributionNum(self, sphereCoords):
        """Creates a dictionary of the three sphere coords that are closest to each
path coord.
        The length of this dictionary will be the distribution score assigned to the
combination.
        A higher distribution score means the path of the combination will better
cover the entire sphere,
        thus the average orientation will be closer to zero and create a better
simulation."""
        octants = self.__splitSphere(sphereCoords) #splits spheres into octants and
returns dictionary with each octant separated

```

```
113         #Every time the path passes between a new segment, the three vertices are
    added as keys.
114         #The value is a list of the indexes of the pathCoords to denote the how long
    it takes to repeat the entire path
115         pathMap = {}
116         repeatTime = -1
117 
118         for pathRow in self.pathCoords:
119             pathOctant = self.__getPathOctant(pathRow) #determines what octant this
    set of path coordinates is in
120             sphereCoordsSplit = octants[pathOctant] #retrieves sphere coordinates in
    the octant that this set of path coordinates is in
121             distDict = {}
122             repeatTime += 1
123 
124             for sphereRow in sphereCoordsSplit:
125                 dist = self.__getDistanceBetween(pathRow, sphereRow) #gets distance
    between two points
126                 distDict[sphereRow] = dist #Updates dictionary (key:sphere
    coordinates, value: distance between path coordinate and that sphere coordinate)
127 
128             rankedDist = sorted(distDict.items(), key=lambda x:x[1]) #takes
    dictionary of distances, sorts them, and makes it into a list of tuples
129             segmentVertices = (rankedDist[0][0], rankedDist[1][0], rankedDist[2][0])
    #the three closest sphere coordinates are the vertices of the segment that the path
    coordinate is in
130 
131             pathMap[segmentVertices] = pathMap.get(segmentVertices, []) +
    [repeatTime] # If the path has not passed through this segment, it creates a key and
    assigns the value as the first time the path passed through the segment
132 
133         return(len(pathMap))
134 
135     def getDistribution(self):
136         #get sphere coords
137         Xsphere, Ysphere, Zsphere = self.__createSphere()
138         sphereCoords = list(zip(Xsphere, Ysphere, Zsphere))
139         score = self.__getDistributionNum(sphereCoords)
140         return score
141 
142     """
143     Distribution Figure Types:
144         Distribution     --> Shadow
145         createPathFig       --> Path
146         createSphereFig     --> Points
147         createPathShadowFig --> Path, Shadow
148         createCombinedFig   --> Path, Shadow, Points
149     """
150     def createPathShadowFig(self, mode='save', separated = False, title = True,
    legend = True):
151         disScore = self.getDistribution()
152         legTitle = 'Distribution: ' + str(disScore)
153         if separated:
154             fig = plt.figure(figsize=plt.figaspect(0.4)) #Separates Shadow and Path
155             plt.xlim(-1.0, 1.0)
156             plt.ylim(-1.0, 1.0)
157 
158             if title:
159                 fig.suptitle(self.ID + '  --  ' + 'Path and Shadow')
160 
```

```
            ax = fig.add_subplot(1, 2, 1, projection='3d')
            ax.plot(self.x, self.y, self.z, label = "Acceleration Vector Path", linewidth = 1) #Plots Path
            ax.legend()

            ax = fig.add_subplot(1, 2, 2, projection='3d')
            ax.plot(self.x, self.y, zs=-1, zdir='z', label='Projection in (x,y)') #Plots Shadow on x,y plane
            ax.plot(self.x, self.z, zs=1, zdir='y', label='Projection in (x,z)') #Plots Shadow on x,z plane
            ax.plot(self.y, self.z, zs=-1, zdir='x', label='Projection in (y,z)') #Plots Shadow on y,z plane


            if legend:
                ax.legend(title=legTitle, loc='lower left')

        else:
            fig = plt.figure(figsize=plt.figaspect(0.85))
            plt.xlim(-1.0, 1.0)
            plt.ylim(-1.0, 1.0)

            if title:
                fig.suptitle(self.ID + '  --  ' + 'Path and Shadow')

            ax = fig.add_subplot(1, 1, 1, projection='3d') #Combines Shadow and Path (default)
            ax.plot(self.x, self.y, self.z, label = "Acceleration Vector Path", linewidth = 1) #Plots Path
            ax.plot(self.x, self.y, zs=-1, zdir='z', alpha = 0.3, label='Projection in (x,y)') #Plots Shadow on x,y plane
            ax.plot(self.x, self.z, zs=1, zdir='y', alpha = 0.3, label='Projection in (x,z)')  #Plots Shadow on x,z plane
            ax.plot(self.y, self.z, zs=-1, zdir='x', alpha = 0.3, label='Projection in (y,z)') #Plots Shadow on y,z plane

            if legend:
                ax.legend(title=legTitle, loc='lower left')

        if mode == 'save':
            plt.savefig(self.saveFile + '-path.png')
        elif mode == 'show':
            plt.show()
        else:
            plt.savefig(self.saveFile + '-path.png')
            plt.show()

    def createCombinedFig(self, mode='save', separated = False, title = True, legend = True):
        pointSize = np.ones(self.num_points) * 2
        Xsphere, Ysphere, Zsphere = self.__createSphere()

        if separated:
            fig = plt.figure(figsize=plt.figaspect(0.4)) #Separates Shadow and Path
            plt.xlim(-1.0, 1.0)
            plt.ylim(-1.0, 1.0)

            if title:
                fig.suptitle(self.ID + '  --  ' + 'Combined')

```

```
211             ax = fig.add_subplot(1, 2, 1, projection='3d')
212             ax.scatter(Xsphere, Ysphere, Zsphere, s = pointSize, color = 'orange')
 #Plots Sphere points
213             ax.plot(self.x, self.y, self.z, label = "Acceleration Vector Path",
linewidth = 1) #Plots Path
214 
215             if legend:
216                 ax.legend()
217 
218             ax = fig.add_subplot(1, 2, 2, projection='3d')
219             ax.plot(self.x, self.y, zs=-1, zdir='z', label='Projection in (x,y)')
#Plots Shadow on x,y plane
220             ax.plot(self.x, self.z, zs=1, zdir='y', label='Projection in (x,z)')
 #Plots Shadow on x,z plane
221             ax.plot(self.y, self.z, zs=-1, zdir='x', label='Projection in (y,z)')
#Plots Shadow on y,z plane
222 
223             if legend:
224                 ax.legend()
225 
226         else:
227             fig = plt.figure(figsize=plt.figaspect(0.85))
228             plt.xlim(-1.0, 1.0)
229             plt.ylim(-1.0, 1.0)
230 
231             if title:
232                 fig.suptitle(self.ID + '  --  ' + 'Combined')
233 
234             ax = fig.add_subplot(1, 1, 1, projection='3d') #Combines Shadow and Path
(default)
235             ax.scatter(Xsphere, Ysphere, Zsphere, s = pointSize, color = 'goldenrod')
#Plots Sphere points
236             ax.plot(self.x, self.y, self.z, label = "Acceleration Vector Path",
linewidth = 1) #Plots Path
237             #ax.plot(self.x, self.y, zs=-1, zdir='z', alpha = 0.3, label='Projection
in (x,y)') #Plots Shadow on x,y plane
238             #ax.plot(self.x, self.z, zs=1, zdir='y', alpha = 0.3, label='Projection
in (x,z)')  #Plots Shadow on x,z plane
239             #ax.plot(self.y, self.z, zs=-1, zdir='x', alpha = 0.3, label='Projection
in (y,z)') #Plots Shadow on y,z plane
240 
241             if legend:
242                 ax.legend()
243 
244         if mode == 'save':
245             plt.savefig(self.saveFile + '-combinedPathSphere.png')
246         elif mode == 'show':
247             plt.show()
248         else:
249             plt.savefig(self.saveFile + '-combinedPathSphere.png')
250             plt.show()
251 
252     def createSphereFig(self, mode='save', title = True):
253         fig = plt.figure(figsize=plt.figaspect(0.85))
254         plt.xlim(-1.0, 1.0)
255         plt.ylim(-1.0, 1.0)
256 
257         if title:
258             fig.suptitle("Number of Points: " + str(self.num_points))
259 
```

```
260         ax = fig.add_subplot(1, 1, 1, projection='3d')
261
262         pointSize = np.ones(self.num_points) * 2
263         Xsphere, Ysphere, Zsphere = self.__createSphere()
264         ax.scatter(Xsphere, Ysphere, Zsphere, s = pointSize, color = 'goldenrod')
265
266         if mode == 'save':
267             plt.savefig(self.saveFile + '-sphere.png')
268         elif mode == 'show':
269             plt.show()
270         else:
271             plt.savefig(self.saveFile + '-sphere.png')
272             plt.show()
273
274     def createShadowFig(self, mode='save', title = True, legend = True):
275         fig = plt.figure(figsize=plt.figaspect(0.85))
276         plt.xlim(-1.0, 1.0)
277         plt.ylim(-1.0, 1.0)
278
279         if title:
280             fig.suptitle(self.ID + '  --  ' + 'Shadow')
281
282         ax = fig.add_subplot(1, 1, 1, projection='3d')
283         ax.plot(self.x, self.y, zs=-1, zdir='z', label='Projection in (x,y)') #Plots
    Shadow on x,y plane
284         ax.plot(self.x, self.z, zs=1, zdir='y', label='Projection in (x,z)')  #Plots
    Shadow on x,z plane
285         ax.plot(self.y, self.z, zs=-1, zdir='x', label='Projection in (y,z)') #Plots
    Shadow on y,z plane
286
287         if legend:
288             ax.legend()
289
290         if mode == 'save':
291             plt.savefig(self.saveFile + '-shadowFig.png')
292         elif mode == 'show':
293             plt.show()
294         else:
295             plt.savefig(self.saveFile + '-shadowFig.png')
296             plt.show()
297
298     def createPathFig(self, mode='save', title = True):
299         fig = plt.figure(figsize=plt.figaspect(0.85))
300         plt.xlim(-1.0, 1.0)
301         plt.ylim(-1.0, 1.0)
302
303         if title:
304             fig.suptitle(self.ID + '  --  ' + 'Path')
305
306         ax = fig.add_subplot(1, 1, 1, projection='3d')
307         ax.plot(self.x, self.y, self.z, label = "Acceleration Vector Path", linewidth
    = 1) #Plots Path
308
309         if mode == 'save':
310             plt.savefig(self.saveFile + '-pathFig.png')
311         elif mode == 'show':
312             plt.show()
313         else:
314             plt.savefig(self.saveFile + '-pathFig.png')
315             plt.show()
```

```
316
317     """
318     Acceleration Figure Types:
319     createVectorFig  --> Figure with individual graphs of x, y, and z raw accel
    values over time
320     createTimeAvgFig --> Figure with individual graphs of time averages of x, y, and
    z over time
321     createMagFig     --> Figure with magnitude over time
322     """
323
324     def formatTime(self, time):
325         startTime = time[0]
326         fTime = []
327         for t in time:
328             fTime.append(t - startTime)
329         return fTime
330
331     def createVectorFig(self, time, mode='save', title=True):
332         #fTime = self.formatTime(time)
333
334         #Plot X Vector
335         fig = plt.figure()
336         ax = fig.add_subplot(1, 1, 1)
337         ax.plot(time, self.x)
338
339         if title:
340             fig.suptitle(self.ID + '  --  ' + 'X Vector')
341
342         if mode == 'save':
343             plt.savefig(self.saveFile + '-XvectorFig.png')
344         elif mode == 'show':
345             plt.show()
346         else:
347             plt.savefig(self.saveFile + '-XVectorFig.png')
348             plt.show()
349
350         #Plot Y Vector
351         fig = plt.figure()
352         ax = fig.add_subplot(1, 1, 1)
353         ax.plot(time, self.y)
354
355         if title:
356             fig.suptitle(self.ID + '  --  ' + 'Y Vector')
357
358         if mode == 'save':
359             plt.savefig(self.saveFile + '-YVectorFig.png')
360         elif mode == 'show':
361             plt.show()
362         else:
363             plt.savefig(self.saveFile + '-YVectorFig.png')
364             plt.show()
365
366         #Plot Z Vector
367         fig = plt.figure()
368         ax = fig.add_subplot(1, 1, 1)
369         ax.plot(time, self.z)
370
371         if title:
372             fig.suptitle(self.ID + '  --  ' + 'Z Vector')
373
```

```
374         if mode == 'save':
375             plt.savefig(self.saveFile + '-ZvectorFig.png')
376         elif mode == 'show':
377             plt.show()
378         else:
379             plt.savefig(self.saveFile + '-ZvectorFig.png')
380             plt.show()
381 
382     def createTimeAvgFig(self, xTimeAvg, yTimeAvg, zTimeAvg, time, mode='save',
    legend=True, title=True):
383         fTime = self.formatTime(time)
384 
385         fig = plt.figure()
386         ax = fig.add_subplot(1, 1, 1)
387         #plt.yscale('log')
388 
389         if title:
390             fig.suptitle(self.ID + '  --  ' + 'Time Average')
391 
392         ax.plot(fTime, xTimeAvg, label='X')
393         ax.plot(fTime, yTimeAvg, label='Y')
394         ax.plot(fTime, zTimeAvg, label='Z')
395 
396         if legend:
397             ax.legend()
398 
399         if mode == 'save':
400             plt.savefig(self.saveFile + '-timeAvgFig.png')
401         elif mode == 'show':
402             plt.show()
403         else:
404             plt.savefig(self.saveFile + '-timeAvgFig.png')
405             plt.show()
406 
407     def createMagFig(self, magnitude, time, mode='save', title=True):
408         fTime = self.formatTime(time)
409 
410         fig = plt.figure()
411         ax = fig.add_subplot(1, 1, 1)
412         plt.yscale('log')
413 
414         if title:
415             fig.suptitle(self.ID + '  --  ' + 'Magnitude')
416 
417         ax.plot(fTime, magnitude)
418 
419         if mode == 'save':
420             plt.savefig(self.saveFile + '-timeMagFig.png')
421         elif mode == 'show':
422             plt.show()
423         else:
424             plt.savefig(self.saveFile + '-timeMagFig.png')
425             plt.show()
426 
```

Supplementary Program 7 – Heatmap Generation (Python)

```
from simulation import Sim
from accelDataProcessing import AccelDataProcesser
from pathVisualization import PathVisualization
import numpy as np

class HeatMapGen:
    innerMin = 0.125
    innerMax = 4
    outerMin = 0.125
    outerMax = 4

    vectorSim = Sim()
    magList = []
    distList = []
    mapOut = open('C:\\MAP_PATH\\MAP_FILENAME', 'w+') #SET MAP PATH AND FILE NAME
    rankOut = open('C:\\RANK_PATH\\RANK_FILENAME', 'w+') #SET RANK PATH AND FILE NAME
    for inV in range(int(innerMin * 8), int(innerMax * 8) + 1):
        magRow = []
        distRow = []
        for outV in range(int(outerMin * 8), int(outerMax * 8) + 1):
            timeArr, xArr, yArr, zArr = vectorSim.gVectorData(0, 10000, float(inV)/8,
float(outV)/8)
            process = AccelDataProcesser(xArr, yArr, zArr, timeArr)
            xTimAvg, yTimAvg, zTimAvg = process.getTimeAvg()

            magArr = []
            for i in range(len(xTimAvg)):
                curX = xTimAvg[i]
                curY = yTimAvg[i]
                curZ = zTimAvg[i]
                curMag = (curX**2 + curY**2 + curZ**2)**0.5
                magArr.append(curMag)

            magRow.append(np.mean(magArr[5000:6000]))

            id = str(inV) + ":" + str(outV)
            pathVis = PathVisualization(id, xArr, yArr, zArr)
            distRow.append(pathVis.getDistribution())
            print(id)
        magList.append(magRow)
        distList.append(distRow)
        magRow = []
        distRow = []
    print(magList)
    print(distList)

    mapOut.write("Magnitude\n")
    for rowI in range(len(magList)):
        for colI in range(len(magList)):
            mapOut.write(str(magList[rowI][colI]) + "\t")
        mapOut.write("\n")

    mapOut.write("\Distribution\n")
    for rowI in range(len(distList)):
        for colI in range(len(distList)):
            mapOut.write(str(distList[rowI][colI]) + "\t")
        mapOut.write("\n")

    mapOut.close()
```

```
59
60      rankOut.write("Combination\tMagnitude\Distribution\n")
61      for rowI in range(len(magList)):
62          for colI in range(len(magList)):
63              inV = float(rowI + 1) / 8
64              outV = float(colI + 1) / 8
65              id = str(inV) + ":" + str(outV) + "\t"
66              rankOut.write(id + str(magList[rowI][colI]) + "\t" + str(distList[rowI]
[colI]) + "\n")
67      rankOut.close()
68
```

Supplementary Program 8 – Temperature Sensor (Python)

```
import os
import glob
import time
import numpy as np

openTime = time.strftime('%b/%d/%Y-%H:%M:%S')

os.system('modprobe w1-gpio')
os.system('modprobe w1-therm')

base_dir = '/sys/bus/w1/devices/'
device_folder = glob.glob(base_dir + '28*')[0]
device_file = device_folder + '/w1_slave'

curData = []

class TEMP:
    def read_temp_raw():
        f = open(device_file, 'r')
        lines = f.readlines()
        f.close()
        return lines

    def read_temp():
        lines = read_temp_raw()
        while lines[0].strip()[-3:] != 'YES':
            time.sleep(0.2)
            lines = read_temp_raw()
        equals_pos = lines[1].find('t=')
        if equals_pos != -1:
            temp_string = lines[1][equals_pos+2:]
            temp_c = float(temp_string) / 1000.0
            temp_f = temp_c * 9.0 / 5.0 + 32.0
            return temp_c

    def startSession():
        d = open('incTemp1.txt', 'a+');
        d.write('\n' + 'Session Start: ' + str(openTime) + '\n\n')

    def curTime():
        return time.strftime('%b/%d/%Y-%H:%M:%S')

    def getTemp():
        if len(curData) < 1:
            startSession()
        curData.append(read_temp())
        d.write(str(read_temp())+ '\t' + curTime() + '\n')
```